

Indices and Ranges in C# 8 with Examples

In this article, I am going to discuss **C# 8 New Features Indices and Ranges** with Examples. Please read our previous article where we discussed [Asynchronous Disposable in C# 8](#) with Examples. We already know about the Range and Indexers. We use them several times in our applications, they provide a short syntax to represent or access a single or a range of elements from the given sequence or collections. Now, we will learn what's newly added in the Range and Indices in C# 8.0. The Range and Indices make the C# syntax simpler and more readable.

Indices and Ranges in C#

Ranges and Indices in C# allow more natural syntax for accessing single items or a range of items from a sequence. So, basically, if you have a collection and you want to access a single element based on the index position, then you need to use Indexer. On the other hand, from the collection, if you want to access a range of elements, then you need to use Range methods. Let us understand this with an example.

```
using System;
using System.Xml.Linq;
namespace Csharp8Features
{
    class IndicesAndRanges
    {
        static void Main(string[] args)
        {
            //Creating a Collection to store list of Countries
            List<string> countries = new List<string>()
            {
                "INDIA",
                "USA",
                "UK",
                "NZ",
                "CANADA",
                "CHINA",
                "NEPAL",
                "RUSIA",
                "SRILANKA",
                "INDONESIA"
            };
            //Accessing Element Based on the Index Position
            Console.WriteLine($"Element at Index Position 1 is {countries[1]}");
            Console.WriteLine($"Element at Index Position 3 is {countries[3]}");
            //Accessing Range of Elements using GetRange Method
            Console.WriteLine("Accessing Range of Elements using Range");
            List<string> countryRange = countries.GetRange(1, 3);
            //Printing the Elements which are accessed by the GetRange Method
            foreach (var item in countryRange)
            {
                Console.Write(item + " ");
            }
            Console.ReadKey();
        }
    }
}
```

```
}  
}  
}
```

Output:

```
Element at Index Position 1 is USA  
Element at Index Position 3 is NZ  
Accessing Range of Elements using Range  
USA UK NZ
```

Indices and Ranges in C# 8

With C# 8, two new types and two new operators are introduced. They are as follows:

Two New Types:

1. **System.Range**: It represents a sub-range of the given sequence or collection.
2. **System.Index**: It represents an index in the given sequence or collection.

Two New Operators:

^ Operator: It is known as the **Index From the End Operator**. It returns an index that is relative to the end of the sequence or collection. It is the most compact and easiest way to find the end elements compare to earlier methods. Please check the below two syntaxes to access the last element of an array using old and new approaches.

Old Method: `var LastValue = myArray[myArray.Length-1]`

New Method: `var LastValue = myArray[^1]`

.. Operator: It is known as the **Range Operator**. It specifies the start and end as its operands of the given range. It is the most compact and easiest way to find the range of the elements from the specified sequence or collection in comparison to earlier methods. Please check the below two syntaxes to access the last element of an array using old and new approaches.

Old Method: `var arr = myArray.GetRange(1, 5);` //This will return five elements from the index position 1

New Method: `var arr = myArray[2..5];` //This will return the elements from Index position 2 to Index Position 5 – 1

These new C# 8 operators make our code cleaner and more readable. If this is not clear at the moment then don't worry, we will try to make you understand with examples.

Example to Understand Indexers in C# 8

The Index is the new feature introduced in C# 8 and it is implemented in **System.Index**, and it is an excellent way to index a collection from the ending. The end index operator **^ (hat operator)** specifies that the index is relative to the end of the sequence. Let us see an example to understand this concept. Please have a look at the below example for a better understanding. Here, we are creating a variable of type Index and storing the

value 4. Then we are accessing elements from the array using that Index variable. Then we are accessing an element from the end of the sequence using the Index From the End Operator.

```
using System;
using System.Threading.Tasks;
namespace Csharp8Features
{
    class IndicesAndRangesDemo
    {
        static void Main(string[] args)
        {
            //Create an Array of Countries
            var countries = new string[]
            {
                "INDIA",
                "USA",
                "UK",
                "NZ",
                "CANADA",
                "CHINA",
                "NEPAL",
                "RUSIA",
                "SRILANKA",
                "INDONESIA"
            };
            //Create a variable of type Index. Index is a value type
            Index i1 = 4;
            Console.WriteLine($"{countries[i1]}"); // Output: "CANADA"
            //You can even directly specify the Integer Index Position as follo
            //Console.WriteLine($"{countries[4]}"); // Output: "CANADA"
            // Index 4 from end of the collection
            Index i2 = ^4;
            Console.WriteLine($"{countries[i2]}"); // Output: "NEPAL"
        }
    }
}
```

Output:



Note: If you are accessing the element from the beginning then you can directly specify the integer index position instead of creating the Index variable. But if you want to access an element from the end of the sequence then you can create an Index variable and then by using the Index From the End Operator, you can access the element.

Understanding Index From Start and Index From End:

Please have a look at the below diagram which shows how Index From Start and Index From End work in C#. Here, you can see, when we use **4** means Index from the start and the value is CANADA, and when we use **^4** means Index from End and the value is NEPAL which you can see in the output window.

```
var countries = new string[]
{
    "INDIA",           //Index From Start    //Index From End
    "USA",             //1                    //^9
    "UK",              //2                    //^8
    "NZ",              //3                    //^7
    "CANADA",          //4                    //^6
    "CHINA",           //5                    //^5
    "NEPAL",           //6                    //^4
    "RUSIA",           //7                    //^3
    "SRILANKA",        //8                    //^2
    "INDONESIA"         //9                    //^1
};
```

Rules for Indexes in C#

Now, let us understand the rules for indexes. Let us consider we have an array of countries. The **0 index** is the same as **countries[0]**. The **^0** index is the same as **countries[countries.Length]**. Note that **countries[^0]** do throw an exception, just as **countries[countries.Length]** does. For any number n, the index **^n** is the same as **countries.Length - n**. In the below example, we are using **^0** and it should throw an exception.

```
using System;
using System.Threading.Tasks;
namespace Csharp8Features
{
    class IndicesAndRangesDemo
    {
        static void Main(string[] args)
        {
            //Create an Array of Countries
            var countries = new string[]
            {
                "INDIA", //Index From Start //Index From End
                "USA", //1 //^9
                "UK", //2 //^8
                "NZ", //3 //^7
                "CANADA", //4 //^6
                "CHINA", //5 //^5
                "NEPAL", //6 //^4
                "RUSSIA", //7 //^3
            };
        }
    }
}
```

```

        "SRILANKA",           //8           //^2
        "INDONESIA"          //9           //^1
    };
    // Index 0 from end of the collection
    Index i2 = ^0;
    Console.WriteLine($"{countries[i2]}"); // Output: "Exception"
}
}
}

```

Output:



Examples to Understand Range in C# 8

The Range is a more natural syntax for specifying or accessing subranges in a sequence. The Range easily defines a sequence of data. It is a replacement for **Enumerable.Range()**, except that the Range, defines the **start and stop** points rather than **start and count**, and it helps you to write more readable code.

A range specifies the start and end of a range. The start of the range is inclusive, but the end of the range is exclusive, meaning the start is included in the range but the end isn't included in the range. The **range** `[0..^0]` represents the entire range, just as `[0..sequence.Length]` represents the entire range.

Bounded Ranges in C#

In the bounded ranges, the lower bound (start index) and the upper bound (end index) are known or predefined.

Syntax: `array[start..end]` // Get items from start until end-1

Let us understand this with an example. The following example creates a subrange with the countries "INDIA", "USA", "UK" and "NZ". It includes countries[0] through countries[3]. The element countries[4] aren't in the range. The following example code is self-explained, so please go through the comment lines for a better understanding.

```

using System;
using System.Threading.Tasks;

```

```

namespace Csharp8Features
{
    class IndicesAndRangesDemo
    {
        static void Main(string[] args)
        {
            //Create an Array of Countries
            var countries = new string[]
            {
                //Index From Start
                "INDIA",           //0
                "USA",             //1
                "UK",              //2
                "NZ",              //3
                "CANADA",          //4
                "CHINA",           //5
                "NEPAL",           //6
                "RUSIA",           //7
                "SRILANKA",        //8
                "INDONESIA"         //9
            };
            //Fetching the First Four Countries
            //It will fetch elements from Index Position 0 to Index Position 4
            var subCountries = countries[0..4]; //INDIA USA UK NZ
            //Printing Sub Country Names
            foreach (var country in subCountries)
            {
                Console.WriteLine(country);
            }
        }
    }
}

```

Output:

```

INDIA
USA
UK
NZ

```

Example: Fetching Elements from the Middle of the Collection using Range

Let us fetch the countries from the UK to CANADA. In our example, the UK Index position is 2, and the CANADA Index Position is 4. So we need to specify 2..5 as shown in the below example.

```

using System;
using System.Threading.Tasks;
namespace Csharp8Features
{

```

```

class IndicesAndRangesDemo
{
    static void Main(string[] args)
    {
        //Create an Array of Countries
        var countries = new string[]
        {
            "INDIA",           //Index From Start
            "USA",              //0
            "UK",               //1
            "NZ",               //2
            "CANADA",           //3
            "CHINA",            //4
            "NEPAL",            //5
            "RUSIA",            //6
            "SRILANKA",         //7
            "INDONESIA"         //8
        };
        //Fetching Elements from the Middle of the Collection
        //It will fetch elements from Index Position 2 to Index Position 4
        var subCountries = countries[2..5]; //UK NZ CANADA
        //Printing Sub Country Names
        foreach (var country in subCountries)
        {
            Console.WriteLine(country);
        }
    }
}

```

Fetching Elements using End Index Operator with Ranges in C# 8

Now, we need to fetch the last three elements from the collection using End Index Operator with Range. For a better understanding, please have a look at the below example. The below example creates a subrange with "RUSIA", "SRILANKA", and "INDONESIA". It includes **countries[^3]** and **countries[^1]**. The end index **countries[^0]** aren't included:

```

using System;
using System.Threading.Tasks;
namespace Csharp8Features
{
    class IndicesAndRangesDemo
    {
        static void Main(string[] args)
        {
            //Create an Array of Countries
            var countries = new string[]
            {
                "INDIA",           //Index From Start
                "USA",              //Index From End
                "UK",               //Index From Start
                "NZ",               //Index From End
                "CANADA",           //Index From Start
                "CHINA",            //Index From End
                "NEPAL",            //Index From Start
                "RUSIA",            //Index From End
                "SRILANKA",         //Index From Start
                "INDONESIA"         //Index From End
            };
        }
    }
}

```



```

        "INDIA",           //0           //^10
        "USA",             //1           //^9
        "UK",              //2           //^8
        "NZ",              //3           //^7
        "CANADA",          //4           //^6
        "CHINA",           //5           //^5
        "NEPAL",           //6           //^4
        "RUSIA",           //7           //^3
        "SRILANKA",        //8           //^2
        "INDONESIA"         //9           //^1
    };
    var allCountries = countries[..]; // contains INDIA through INDONE
    Console.WriteLine("All Countries");
    foreach (var country in allCountries)
    {
        Console.Write($"{country} ");
    }

    //Lower Bound is Ommited, so it will start from 0
    var firstPhrase = countries[..5]; // contains INDIA through CANADA
    Console.WriteLine("\nFirst Phrase Countries");
    foreach (var country in firstPhrase)
    {
        Console.Write($"{country} ");
    }
    //Upper Bound is Ommited, so it will end with the Length of the co
    var lastPhrase = countries[6..]; // contains NEPAL through INDONE
    Console.WriteLine("\nLast Phrase Countries");
    foreach (var country in lastPhrase)
    {
        Console.Write($"{country} ");
    }
}
}
}

```

Output:

```

All Countries
INDIA USA UK NZ CANADA CHINA NEPAL RUSIA SRILANKA INDONESIA
First Phrase Countries
INDIA USA UK NZ CANADA
Last Phrase Countries
NEPAL RUSIA SRILANKA INDONESIA

```

Ranges as variables in C#

We can also declare ranges as variables in C#. The following is the syntax:

Range phrase = 1..5;

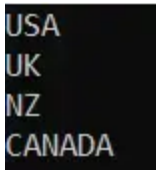
The range. then can be used inside the [] characters as follows:

```
var subCountry= countries[phrase];
```

The following example shows how to use Ranges as variables in C#.

```
using System;
using System.Threading.Tasks;
namespace Csharp8Features
{
    class IndicesAndRanges
    {
        static async Task Main(string[] args)
        {
            var countries = new string[]
            {
                "INDIA",           //Index From Start    //Index From End
                "USA",             //1
                "UK",              //2
                "NZ",              //3
                "CANADA",          //4
                "CHINA",           //5
                "NEPAL",           //6
                "RUSIA",           //7
                "SRILANKA",        //8
                "INDONESIA"        //9
            };
            Range phrase = 1..5;
            var subCountries = countries[phrase];
            foreach (var country in subCountries)
            {
                Console.WriteLine($"{country} ");
            }
        }
    }
}
```

Output:



Note: Not only do arrays support indices and ranges but you can also use indices and ranges with string, Span<T>, or ReadOnlySpan<T>.

Range with Strings in C#:

Ranges in C# allow the creation of substrings by using the indexer. Please have a look at the below example for a better understanding.

```
using System;
using System.Threading.Tasks;
namespace Csharp8Features
{
    class IndicesAndRanges
    {
        static async Task Main(string[] args)
        {
            var helloWorldStr = "Hello, World!";
            var hello = helloWorldStr[..5]; // Take 5 from the begin
            Console.WriteLine(hello); // Output: Hello
            var world = helloWorldStr[7..]; // Skip 7
            Console.WriteLine(world); // Output: World!
        }
    }
}
```

Output:

```
Hello
World!
```

Or you can write it like the following:

```
using System;
using System.Threading.Tasks;
namespace Csharp8Features
{
    class IndicesAndRanges
    {
        static async Task Main(string[] args)
        {
            var helloWorldStr = "Hello, World!";
            var hello = helloWorldStr[..5]; // Take 5 from the begin
            Console.WriteLine(hello); // Output: Hello
            var world = helloWorldStr[^6..]; // Take the last 6 characters from the end
            Console.WriteLine(world); // Output: World!
        }
    }
}
```

Output:

```
Hello
World!
```

Ranges Foreach loops in C#

Range with IEnumerable Example. Ranges implement IEnumerable<int>, which allows the iteration over a sequence of data

```
using System;
using System.Threading.Tasks;
namespace Csharp8Features
{
    class IndicesAndRanges
    {
        static async Task Main(string[] args)
        {
            var countries = new string[]
            {
                "INDIA",           //Index From Start    //Index From End
                "USA",              //0                    //^10
                "UK",               //1                    //^9
                "NZ",               //2                    //^8
                "CANADA",           //3                    //^7
                "CHINA",            //4                    //^6
                "NEPAL",            //5                    //^5
                "RUSSIA",           //6                    //^4
                "SRILANKA",         //7                    //^3
                "INDONESIA",        //8                    //^2
            };
            foreach (var firstFourCountries in countries[1..5])
            {
                Console.WriteLine($"{firstFourCountries} ");
            }
        }
    }
}
```

Output:



```
USA
UK
NZ
CANADA
```

In the next article, I am going to discuss [Null-Coalescing Assignment ??= Operator in C# 8](#) with Examples. Here, in this article, I try to explain **Indices and Ranges in C# 8** with Examples. I hope you enjoy this Indices and Ranges in C# 8 with Examples article.